Creating Markdown Files for Codex

https://gemini.google.com/app/a03b38970de16c49

User prompt: Can you help how to create .md file for codex, with some exapmples

Response: Creating a `.md` **(Markdown)** file for OpenAI's Codex (or general LLM prompt engineering, instructions, and documentation) is a great way to provide context, coding standards, or system prompts.

Here is a complete guide on how to create one, what goes inside it, and practical examples you can use right away.

1. How to Create a `.md` File

You can create a Markdown file in any text editor or your code editor:

1. Open your code editor (like **VS Code**).
2. Create a new file and name it with a `.md` extension (e.g., `codex-instructions.md`, `prompt.md`, or `README.md`).
3. Write your content using standard Markdown syntax (headings, bullet points, code blocks).

2. Examples of `.md` Files for Codex

Depending on what you are trying to achieve, here are three common types of `.md` files you might create for Codex/LLM workflows:

Example 1: Coding Standards / System Instructions (`codex-rules.md`)

Use this when you want Codex to follow strict guidelines regarding language versions, formatting, or frameworks.

```
# Codex Coding Guidelines & System Instructions

You are an expert full-stack developer assisting with a Next.js and TypeScript project. Follow these rules strictly in a

## 1. Tech Stack
- **Framework:** Next.js 14 (App Router)
- **Language:** TypeScript (strict mode enabled; avoid using `any`)
- **Styling:** Tailwind CSS

## 2. Code Style
- Write functional, modular React components.
- Always use asynchronous error handling (`try/catch` blocks inside Server Actions or API routes).
- Add clean JSDoc comments for complex helper functions.

## 3. Example Component Structure
```typescript
import React from 'react';

interface ButtonProps {
  label: string;
  onClick: () => void;
  variant?: 'primary' | 'secondary';
}

export default function CustomButton({ label, onClick, variant = 'primary' }: ButtonProps) {
  const baseStyles = "px-4 py-2 rounded-lg font-medium transition-colors";
  const variantStyles = variant === 'primary'
    ? "bg-blue-600 text-white hover:bg-blue-700"
    : "bg-gray-200 text-gray-800 hover:bg-gray-300";

  return (
    <button onClick={onClick} className={`${baseStyles} ${variantStyles}`}>
      {label}
    </button>
  );
}


---

### Example 2: Few-Shot Prompt Template (`sql-generator.md`)
Use this when you want to feed Codex a pattern or specific examples so it learns how to translate natural language into

```markdown
# Natural Language to SQL Converter
```

````
Your task is to convert English requests into PostgreSQL queries based on the schema below.

## Database Schema
- **users** (`id`, `name`, `email`, `created_at`)
- **orders** (`id`, `user_id`, `total_amount`, `status`, `order_date`)

---

## Examples

### Example 1
- **User Request:** Find all active orders placed in 2026.
- **SQL Query:**
```sql
SELECT * FROM orders
WHERE status = 'active'
  AND EXTRACT(YEAR FROM order_date) = 2026;
````

Example 2

- **User Request:** Get the total revenue generated by user with ID 42.
- **SQL Query:**

````
SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE user_id = 42;


---

### Example 3: Project Documentation / Context File (`project-context.md`)
Codex models work best when they understand the broader picture of your workspace.

```markdown
# Project Context: Inventory Management API

## Overview
This repository contains a Node.js Express backend designed to track warehouse inventory levels in real-time.

## Core Architecture
- **Entry Point:** `src/index.ts`
- **Database ORM:** Prisma with PostgreSQL
- **Authentication:** JWT Bearer tokens handled via middleware (`src/middleware/auth.ts`)

## Key Guidelines for Codex
- When writing new API endpoints, always attach the `verifyToken` middleware.
- Return standardized JSON responses:
  ```json
  {
    "success": true,
    "data": {},
    "error": null
  }


---

## Tips for Writing Effective Markdown for LLMs
* **Use Clear Headings (`#`, `##`, `###`):** Codex uses headers to parse hierarchical instructions and segment rules.
* **Leverage Fenced Code Blocks (` ``` `):** Always wrap code examples in backticks and specify the language (` ```pytho
* **Keep Rules Explicit:** Instead of saying *"write clean code"*, specify *"never use `any` in TypeScript"* or *"always

Would you like help writing a specific `.md` file tailored to a project or language you are working on?
````

User prompt: I need help to create .md for current project it would be helpful future usage for team members

Response: I will now generate a comprehensive Markdown template for your current project's setup guide and documentation, designed to help onboard future team members effectively.

````

User prompt: let say, this .md file will be used by agent to repond to the user request through prompt?

Response: Yes, exactly! If that `.md` file is used by an AI agent (or IDE extension like GitHub Copilot Workspace, Cursor, or a custom LLM system prompt), it functions as a **System Instruction, Context Document, or Agent Persona file**.

Instead of just onboarding human team members, feeding a `.md` file into an AI agent's prompt pipeline ensures the agent acts with project-specific awareness. Here is how it changes how the agent responds, along with a tailored template optimized specifically for agent instruction.

How an AI Agent Uses a `.md` File

1. **Context Grounding:** The agent reads the file at initialization to understand the tech stack, directory structure, and guardrails, preventing it from suggesting outdated libraries or incorrect syntax.
2. **Behavioral Constraint:** You can explicitly tell the agent *how* to reply (e.g., *"Always provide unit tests with code"* or *"Never suggest using class components in React"*).
3. **Consistency:** Multiple team members using the same agent configuration will get uniform code outputs that match your team's exact standards.

Example: A `.md` File Optimized for AI Agent Instructions (`agent-rules.md`)

If you are setting up a Markdown file specifically to guide an AI agent or coding assistant, structure it like this:

```
# AI Agent Persona & Instructions

## 1. Role & Objective
You are an expert AI software engineer assisting development teams. Your goal is to write clean, secure, and production-

## 2. Mandatory Rules & Guardrails
- **No Deprecated APIs:** Never use legacy functions or libraries listed in the deprecation backlog.
- **Error Handling:** Every API route or database query must include explicit `try/catch` or Result-type error handling.
- **Testing:** When writing a new feature, always include a corresponding unit test using Jest/Vitest.
- **Code Style:** Keep functions small and modular. Add concise JSDoc comments only for complex logic.

## 3. Tech Stack Reference
- **Frontend:** React 18, TypeScript, Tailwind CSS
- **Backend:** Node.js, Express, Prisma ORM
- **Database:** PostgreSQL

## 4. Response Format
When answering user coding requests, structure your response as follows:
1. **Brief Explanation:** 1–2 sentences explaining the approach.
2. **Code Implementation:** Clean, properly indented code blocks with language specifiers.
3. **Verification/Testing:** How to test or verify the code snippet works.
```

Immediate Next Steps

1. Create a file named `AGENT_RULES.md` or `.github/copilot-instructions.md` (depending on the tool you use).
2. Paste your project's specific conventions, architectural boundaries, and preferred response format into it.
3. Reference or load this file in your agent setup or system prompt configuration.

User prompt: Is there any syntax needs to follow in .md file?

Response: Yes! While Markdown (`.md`) is designed to be human-readable plain text, AI agents and parsers rely on specific structural syntax to properly understand hierarchy, rules, and code blocks.

Here are the core Markdown syntax rules you should follow so an AI agent parses your instruction file accurately:

1. Document Hierarchy (Headings)

Use hash symbols (`#`) to create a strict hierarchy. AI agents use these headers to segment instructions (e.g., separating tech stack from coding rules).

- `#` : **H1** — Use once at the very top for the main title (e.g., `# Project Agent Instructions`).
- `##` : **H2** — Use for major sections (e.g., `## 1. Tech Stack`, `## 2. Coding Rules`).
- `###` : **H3** — Use for subsections under H2.

2. Code Blocks (Fenced Backticks)

Always wrap code snippets, configuration files, or schema examples in triple backticks (```) and specify the language identifier right after them. This tells the AI: *"Treat this literal text as code, not instructions."*

```
```typescript
// Example of how you want code written
const add = (a: number, b: number): number => a + b;


### 3. Lists for Rules and Constraints
AI models excel at following bulleted or numbered lists. Use standard markers:
* **Unordered Lists (`*` or `-`)**: Best for general features, tech stack items, or non-sequential bullet points.
* **Numbered Lists (`1.`, `2.`, `3.`)**: **Crucial** when sequence matters (e.g., step-by-step instructions or ordered r

### 4. Emphasis for Strict Rules
Use bolding (`**`) to draw the AI's attention to critical constraints or negative rules (things it *must not* do).
* *Example:* `* **Never** use the `any` type in TypeScript files.`
* *Example:* `* **Always** include a try/catch block in API routes.`

### 5. Blockquotes for Core Personas
If you want to set a high-level persona or rule that applies globally, you can use blockquotes (`>`):
```markdown
> **Core Directive:** Prioritize code readability and strict type safety over clever or concise one-liners.
```

Clean Example Template Incorporating All Syntax Rules

```
# Agent Instructions: Frontend React App

## 1. Tech Stack Constraints
- **Framework:** React 18 with TypeScript
- **Styling:** Tailwind CSS only (do not use inline styles or vanilla CSS files).

## 2. Strict Rules
* **Never** use class components; write functional components exclusively.
* **Always** handle loading and error states for any asynchronous API calls.

## 3. Example Component Format
```tsx
export default function UserCard({ name }: { name: string }) {
  return <div className="p-4 bg-white rounded-lg shadow">{name}</div>;
}
```

User prompt: http://skill.md/ i found this for code review can you expalin this

Response: The `skill.md` format (or a `SKILL.md` file) is an open standard used by AI coding agents and CLI tools (like Claude Code, Cursor, and Codex CLI) to package specialized instructions into a reusable "skill".

Instead of typing out long context rules every time, you drop a single `SKILL.md` file into your project or global directory, and the AI automatically detects and uses it when triggered.

How a `SKILL.md` File is Structured

Every valid `SKILL.md` file consists of two parts:

1. **YAML Frontmatter (`--- ... ---`):** Metadata at the very top that tells the AI *what* the skill is and *when* to activate it based on your prompt.
2. **Markdown Body (`# ...`):** Step-by-step instructions telling the AI how to execute the task.

Example: A Code Review Skill (`SKILL.md`)

If you downloaded or created a code review skill like the one from the skills marketplace, it looks like this:

```
---
name: code-review
description: Pre-PR code review for branch changes. Use when the user wants to review changes before opening a PR, check
effort: high
---

# Code Review Instructions

When the user requests a code review, follow these systematic steps:

## 1. Diff Analysis
- Check the git modifications against the base branch.
```
```

```
- Identify changed files, added logic, and removed sections.

## 2. Evaluation Criteria
Evaluate the code across these dimensions:
- **Correctness & Edge Cases:** Look for off-by-one errors, null checks, and unhandled promises.
- **Security:** Check for input validation gaps, injection risks, or exposed secrets.
- **Performance:** Flag unnecessary loops, heavy memory usage, or missing database indexes.

## 3. Response Format
Provide feedback using categorized priority labels:
- 🔴 **[blocking]** - Must fix before merge (bugs, security flaws).
- 🟡 **[important]** - Should fix (code smells, missing error boundaries).
- 🟢 **[nit]** - Optional style or formatting suggestions.
```

Why Use `SKILL.md` Files?

- **Cross-Tool Portability:** A single `SKILL.md` file works across multiple AI coding assistants without needing to rewrite configuration rules for each tool.
- **On-Demand Activation:** Because of the `description` field in the frontmatter, the AI only reads and applies the instructions when your request matches its intended trigger (e.g., saying *"review my code"* loads the code-review skill).
- **Modular Setup:** Instead of having one massive, bloated system instruction file, you can separate different tasks into individual skill folders (e.g., a `code-review/SKILL.md`, a `testing/SKILL.md`, etc.).

Sources

1. SKILL.md Format Specification: Complete YAML Frontmatter
2. code-review - Agent Skills Marketplace for Claude, Codex & ChatGPT
3. SKILL.md, the open format for Claude, Cursor, and MCP skills

---

User prompt: What is the use of agent file

Response: An **agent file** (often named `agent.md`, `.agentignore`, or configured through tool-specific agent manifests like `AGENT.md`) serves as the core operational brain and behavioral boundary for an AI assistant working inside a specific workspace or repository.

Its primary purposes include:

1. **Defining Operational Boundaries & Capabilities:** It tells the agent what it *is* and what it *can* or *cannot* do. For example, it can restrict an agent from modifying production database files, writing tests in unsupported frameworks, or touching specific sensitive directories.
2. **Establishing Repository Context & Architecture Map:** Instead of the agent guessing how your project is structured, an agent file points it directly to core architectural paths (e.g., *"API routes are in `src/api`, components are in `src/components`"*), saving tokens and preventing hallucinated file paths.
3. **Standardizing Autonomous Workflows:** When using advanced AI coding agents that execute multi-step tasks autonomously (like writing code, running tests, and creating git commits), the agent file dictates the step-by-step lifecycle it must follow before declaring a task complete.
4. **Persistent Environment Configuration:** It acts as a permanent configuration layer so that every time you spin up a fresh agent session in that project, it instantly adopts your team's specific working style, testing commands, and deployment rules without requiring manual prompting.

User prompt: Can you give example gor agent and skill

Response: To see how an **Agent file** and a **Skill file** work together, it helps to look at their distinct roles:

- **The Agent File (`AGENTS.md`)** is the **permanent roadmap and rulebook** for your project. It stays loaded all the time so the AI always knows your tech stack, folder layout, and core safety constraints.
- **The Skill File (`SKILL.md`)** is a **on-demand playbook** for a specific task. It stays tucked away until you trigger a specific action (like running a code review or building a database migration).

Example 1: The Agent File (`AGENTS.md`)

Placed at the root directory of your repository (e.g., `AGENTS.md`)

```
# Project Guidelines for AI Agents

## 1. Overview
This repository contains a full-stack task management application built for internal operations.

## 2. Tech Stack & Tools
- **Frontend:** Next.js (App Router), TypeScript, Tailwind CSS
- **Backend/DB:** Node.js, Prisma ORM, PostgreSQL
```

```
- **Commands:**
  - Install: `npm install`
  - Run tests: `npm test`
  - Type check: `npm run typecheck`

## 3. Directory Structure
- `src/app/`: Next.js pages and API route handlers
- `src/components/`: Reusable UI elements
- `src/server/`: Prisma database clients and server actions

## 4. Core Constraints
- Never introduce a new external production dependency without explicit confirmation.
- Always write explicit TypeScript types (avoid using `any`).
- Always run `npm run typecheck` mentally or via terminal tools before declaring code changes complete.
```

Example 2: The Skill File (`SKILL.md`)

Placed in a specific skill directory (e.g., `.skills/code-review/SKILL.md` or ~/.codex/skills/code-review/SKILL.md)

```
---
name: code-review
description: Pre-PR code review protocol. Use when the user asks to review changes, audit a branch, or check code qualit
---

# Code Review Instructions

When this skill is triggered, execute the following workflow step-by-step:

## Step 1: Diff and Scope Analysis
- Inspect all modified files in the current workspace or branch diff.
- Verify whether any changes violate the rules specified in `AGENTS.md` (such as unauthorized package installs or missin

## Step 2: Evaluation Matrix
Check the code against these three risk criteria:
1. **Security & Data Safety:** Look for exposed credentials, unescaped SQL queries, or missing input validation.
2. **Error Handling:** Ensure asynchronous operations contain proper `try/catch` wrappers.
3. **Performance:** Flag any nested loops or expensive rendering logic.

## Step 3: Structured Reporting
Format your feedback clearly using these priority tags:
- 🔴 **[Blocking]**: Critical bugs or security vulnerabilities that must be fixed immediately.
- 🟡 **[Important]**: Missing error boundaries or code style violations.
- 🟢 **[Nit]**: Minor naming improvements or optional cleanups.
```

How They Work Together

1. When you open your project, the AI reads **`AGENTS.md`** so it automatically knows you are writing TypeScript/Next.js code and respects your build commands.
2. When you type *"Can you review my changes before I open a PR?"*, the AI detects the keyword match in the **`code-review`** skill's description, loads the **`SKILL.md`** file, and executes the structured review process.

Sources

1. What Is SKILL.md? A Complete Guide to AI Agent Skills
2. AGENTS.md vs SKILL.md vs CLAUDE.md vs .cursorrules… - Agensi

---

User prompt: Does the above agent and skill md files work for codex?

Response: **Yes, absolutely.** The `AGENTS.md` and `SKILL.md` (Agent Skills) formats are fully native to Codex and function as its primary configuration ecosystem.

Codex relies on these files to structure how it reads your project and executes tasks:

1. How Codex Uses `AGENTS.md` (The Always-On Context)

- **Where Codex looks for it:** Codex reads `AGENTS.md` automatically. It checks your global setup (`~/.codex/AGENTS.md`) and your repository root (`./AGENTS.md`).
- **What it does:** Codex reads this file at the start of every session. It uses the build commands, folder layouts, and styling boundaries you outlined so it doesn't have to guess how your project works.

2. How Codex Uses `SKILL.md` (The On-Demand Playbook)

- **Where Codex looks for it:** Codex looks inside your local project skills folder (`.codex/skills/` or `.agents/skills/`) or your global personal directory (`~/.codex/skills/`).
- **What it does:** Codex uses a technique called **progressive disclosure**. It only reads the short metadata descriptions of your skills initially. When your prompt matches a skill's description (or you invoke it explicitly), Codex loads the full body of the `SKILL.md` file into context to execute that specific workflow.

Sources

1. Codex CLI Skills & AGENTS.md Setup Guide 2026 | Install… - Agensi
2. What Is AGENTS.md? The Agent Instruction File Explained - AQ
3. How to Build Custom Commands for Codex Using Agent Skills
